

Group 16: Soccer Players Performance Final Report

SP26: CS-133 Sec 01 - Data Visualization

Team Members:

Hrishikesh Giri

Srihan Cheemangunta

Yeqing Xu

Instructor: Jessica Huynh-Westfall

Date: April 30, 2026

1. Abstract

This project analyzes match-by-match player performance data from the Fantasy Premier League dataset (2016–17 to 2025–26 seasons), sourced from the open-source repository maintained by Vaastav Anand on [GitHub](#). The dataset captures player-level statistics across each gameweek of the 2024-25 and 2025-26 Premier League seasons (49,958 player-gameweek rows across 54 columns), including performance metrics like goals, assists, ICT index, and contextual factors such as home/away status and opponent difficulty. Our main goal is to explore whether historical match-by-match data can support evidence-based player valuation and performance prediction in soccer.

The visualization strategy proceeds in three stages: (1) static categorical and distributional plots (Seaborn, Plotly) to characterize how contextual factors shape player output; (2) an interactive Panel dashboard for a simple, budget-constrained lineup optimization; and (3) a tuned Random Forest regressor. The ML pipeline reaches a test RMSE of 2.83 and R^2 of 0.08 with modest predictive power. We have three takeaways: (a) the model captures average tendencies but cannot anticipate big hauls or disasters, so it lives in a "safe middle" range of 2-5 points; (b) playing time, FPL price, and ICT involvement matter more; and (c) training within a single season produced better test performance than training across seasons (RMSE 2.83 vs 2.92), showing that cross-season distribution shift in scoring rules and tactics outweighs the benefit of more training rows.

2. Background & Problem Definition

Modern soccer performance analysis is now studied through a wide range of indicators such as minutes, expected goals, ICT involvement, and opponent difficulty. These inform decisions made by managers, scouts, fantasy sports players, and data analysts. Several members of our team are soccer fans, and we are interested in how player and team performance can be interpreted before the match outcome is known.

2.1 Dataset Description

We use the Fantasy Premier League dataset at [vaastav/Fantasy-Premier-League](#), which mirrors the official FPL API season by season. The repository covers seasons 2016-17 through 2025-26. For this project, we focus on the two most recent seasons (2024-25 and 2025-26) because they share the same modern feature schema and reflect current Premier League rules.

Each row in the gameweek-merged tables represents one player in one gameweek. After loading,

the combined dataset has 49,958 rows $\times$ 54 columns (27,605 rows from 2024-25 and 22,353 rows from 2025-26 GW1–29). Key features include:

1. Outcome: total_points (FPL points scored that gameweek, and this is our prediction target)
2. Performance: minutes, goals_scored, assists, bonus, clean_sheets, saves, bps
3. Underlying involvement: ict_index, creativity, threat, influence, expected_goals, expected_assists, expected_goal_involvements
4. Contextual: was_home, opponent_team, value (FPL price), position, kickoff_time
5. 2025-26 only (new defensive scoring): defensive_contribution, recoveries, tackles, clearances_blocks_interceptions

We selected this dataset for two reasons. First, it is structurally rich because every game produces 20 or more measurable per-player features, enough to train and evaluate ML models without resorting to outcome leakage. Second, it is public, well-maintained, and regularly updated.

2.2 Problem Statement & Analytical Questions

Problem: Player performance valuation in soccer is complex because outcomes depend on many interacting factors. Yet most fantasy sports decisions, analyses, and some scouting approaches still rely on past point totals or subjective judgment. We aim to investigate whether systematic, evidence-based predictions can be drawn from historical match-by-match data, and whether those predictions are useful enough to support practical decisions.

The five guiding **analytical questions** are:

1. How well can we predict a player's next-game performance using historical statistics?
2. How do contextual factors such as home/away status, opponent difficulty, and player position affect player output?
3. What combination of players is expected to maximize team performance under budget constraints?
4. Which statistics are the strongest early indicators of a breakout player?
5. If a key player becomes unavailable, can we recommend the best replacement?

Connection between visualizations and ML. Each question is answered by a pairing of visualizations and modeling. Distribution and correlation plots show that the target is right-skewed and that several rolling features correlate moderately with total_points, justifying both a tree-based regressor and engineered rolling features. Categorical plots (home/away, opponent difficulty, position) confirm which contextual variables carry a predictive signal. Those

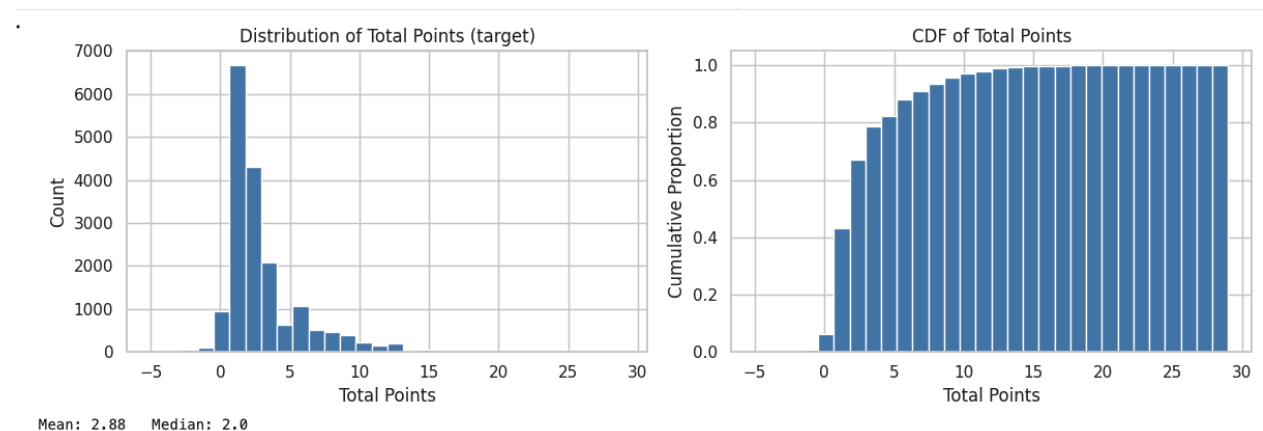
become input features for the ML pipeline. The interactive lineup dashboard directly answers Q3 and demonstrates how visualization can support decision-making under constraints. Finally, the Random Forest feature-importance bar chart translates the trained model's internal weights into a visualization that directly answers Q4 and informs the replacement recommender used for Q5.

3. Data Visualization Strategy & Interactive Component

3.1 Static Visualizations

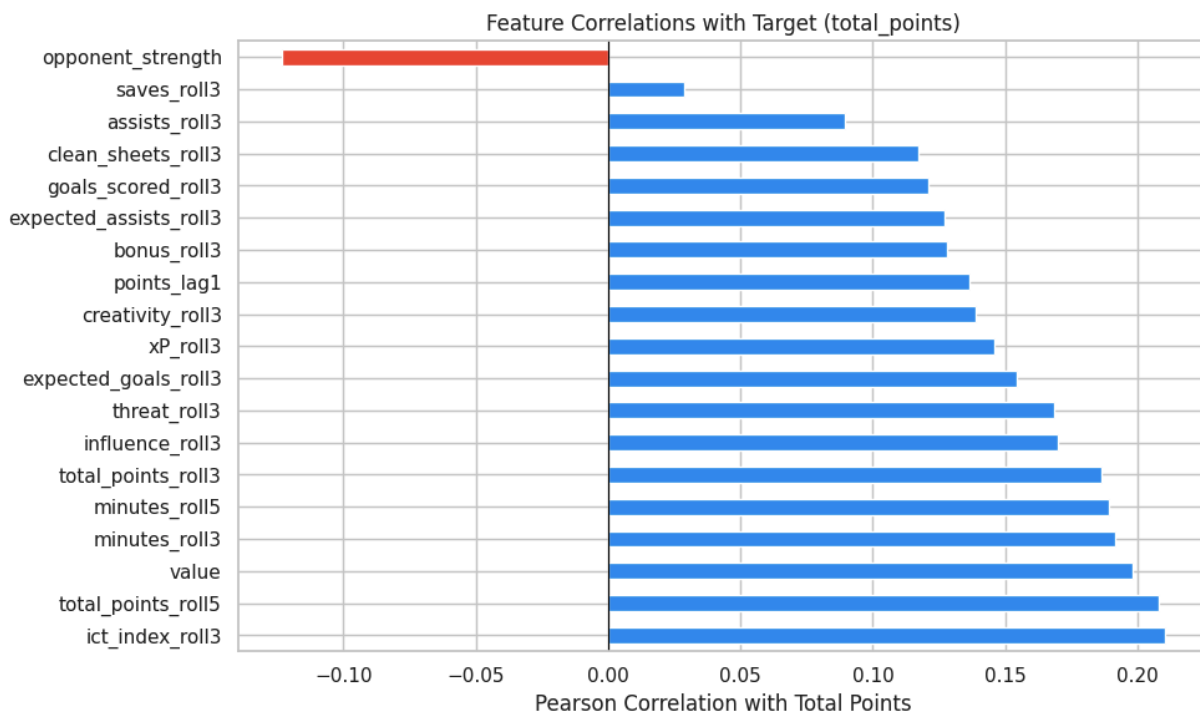
Below are the five visualizations we use to characterize the data, motivate our model choices, and answer Q2, Q4, and parts of Q1. All plots use the cleaned dataset after dropping seven `mng_*` columns, filtering to outfield positions (GK, DEF, MID, FWD), removing zero-minute rows from the modeling set, and engineering 3- and 5-gameweek rolling features (`shift(1)` to prevent leakage).

Visualization 1 — Distribution of `total_points` (histogram + CDF).



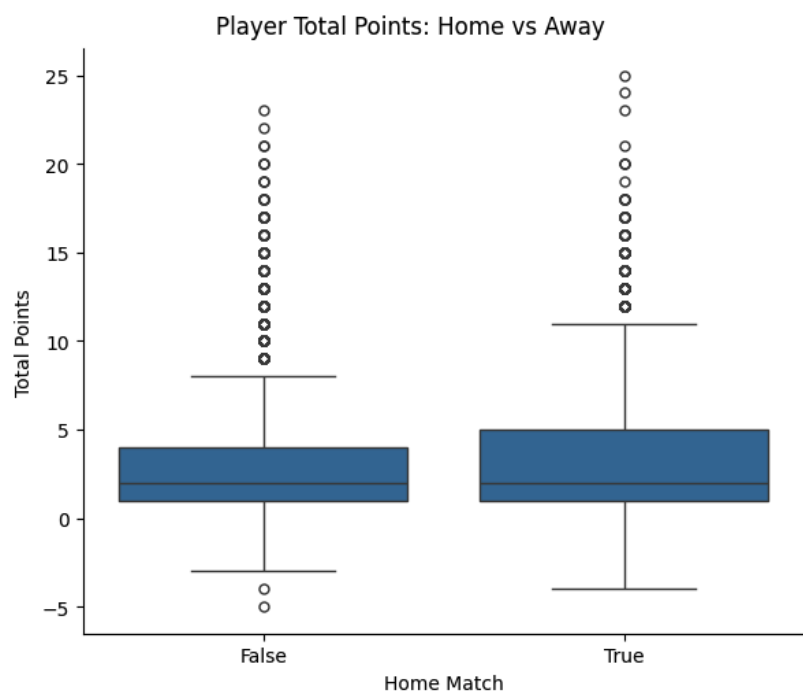
The target is heavily right-skewed. The mean is 2.88, and the median is 2, with 10+ point hauls being rare events. This shape tells us that RMSE will be dominated by the upper tail, and the skew motivates trying both a linear regression and a non-linear ensemble because we want to test whether non-linear feature interactions exist.

Visualization 2 — Feature correlations with `total_points`.



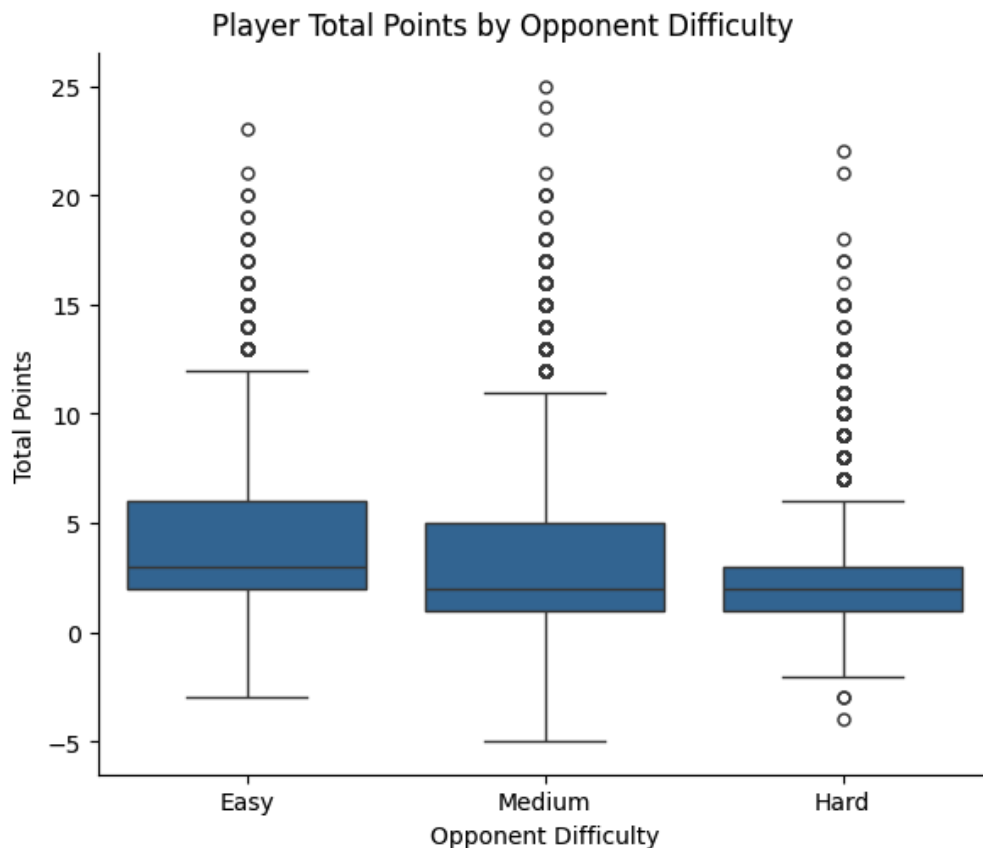
The horizontal bar chart shows the correlation between each numeric rolling feature and the target. Rolling minutes, ICT, and total points show the strongest positive correlation, while opponent_strength is negatively correlated.

Visualization 3 — Player total points by home/away status (categorical box plot).



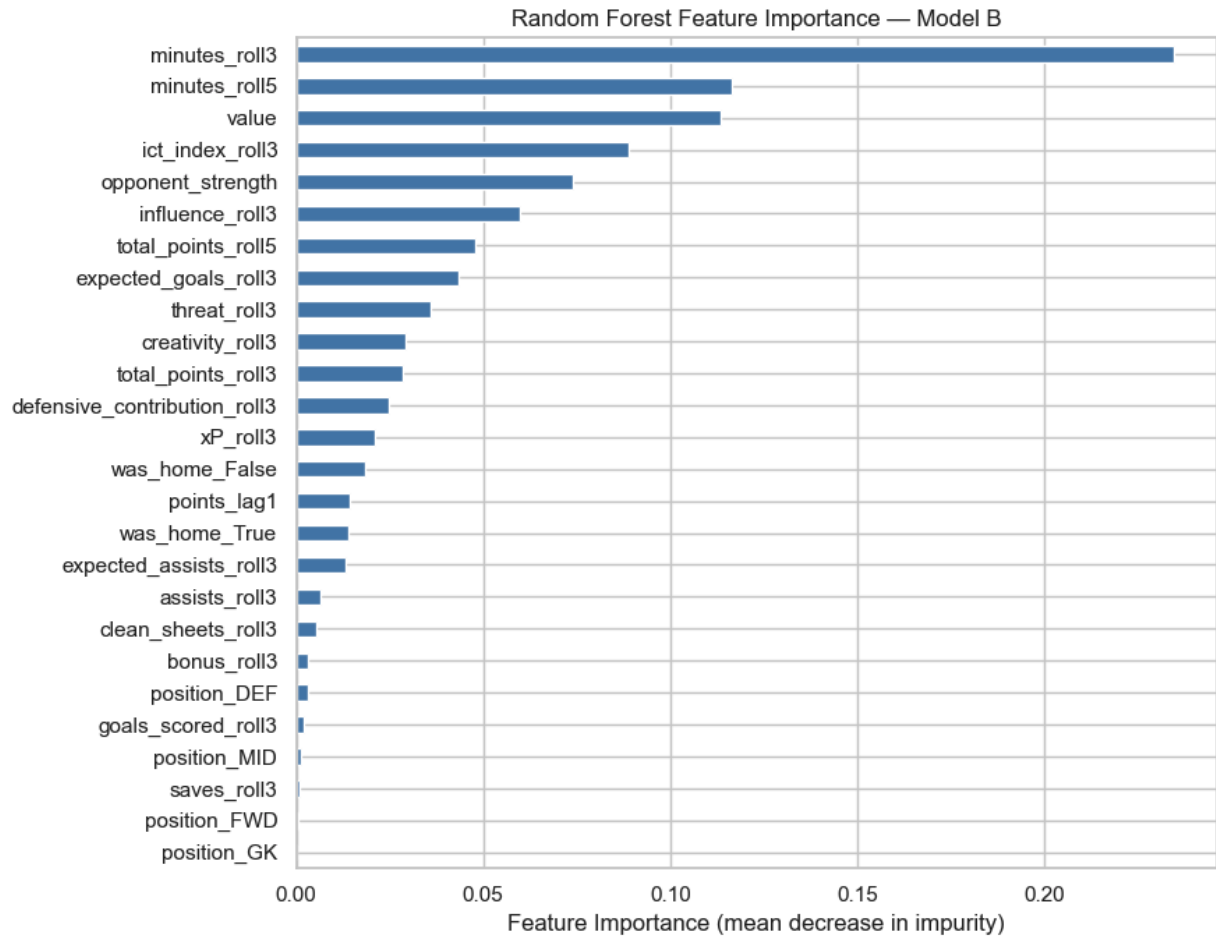
Filtering to active players (play time ≥ 30 minutes), the median of total points sits at 2 points for both groups, but the upper whisker and 75th percentile are clearly higher for home games. This answers part of Q2, that is home advantage is small at the median but visible in the upper tail. `was_home` is a categorical feature in the ML pipeline.

Visualization 4 — Player total points by opponent difficulty (categorical box plot).



We used `apply()` to map opponent strength ratings into Easy/Medium/Hard buckets via the `team.csv` join, and found that opponent difficulty compresses the upper tail more than it shifts the median. Against “Hard” opponents, the 75th percentile drops to 3 points, while against “Easy” opponents, it increases to 6. This confirms that opponent strength is a contextual feature for the ML pipeline.

Visualization 5 — Random Forest feature importance.



This plot answers Q4. The top two features are `minutes_roll3` (0.234) and `minutes_roll5` (0.117), together accounting for about a third of total importance. This shows that playing time is the foundation. `Value` (0.113) and `ict_index_roll3` (0.089) follow. `Total_points_roll5` is only about 0.05, meaning past points alone are not the strongest signal.

Logical connection to ML model selection. Visualization 1 shows a right-skewed target, meaning that RMSE will be dominated by the upper tail, while most rows sit between 1 and 4 points. Visualization 2-4 shows that no single feature dominates, and we need a model that uses many features and that scales them properly, but they do not tell us whether that model should be a linear regression, a random forest, or anything else. That choice will be made later by comparing the actual results. Visualization 5 shows that the engineered rolling features carry the predictive weight, validating the feature-engineering choices made before training.

3.2 Interactive Visualization

The interactive component is a Panel dashboard built with plotly and panel, addressing Q3 (lineup optimization under budget constraints). It is implemented in [P3_interactive.ipynb](#) Question 7.



What it does. The dashboard searches a simplified 5-player lineup (1 GK, 1 DEF, 2 MID, and 1 FWD) within a user-set budget cap and presents the highest-scoring valid combination, along with a scatter plot of player cost vs the chosen performance metric.

User interactions.

- Season dropdown: switch between 2024-25 and 2025-26.
- Budget Cap slider: adjust the total budget from 25 to 55 (unit: tens of millions)
- Metric dropdown: optimize for total points or points per cost
- Min Minutes slider: filter out marginal contributors before optimization.

The scatter plot updates with the selected season, metric, and minutes filter, with players colored by position and a vertical reference line showing the per-player average cost given the budget. The lineup table below shows the best valid combination found and reports the total cost and total score. The interactive dashboard lets a user actually answer the question like “Given a £40M budget, what is the best 5-player lineup?” and users can see the trade-off between cost and output.

4. Machine Learning Pipeline & Evaluation

4.1 Data Preparation and Pipeline

[Describe how you created your train/test splits. Explain your preprocessing steps including scaling, encoding, balancing, and feature engineering. Detail how your pipeline follows the practices demonstrated in course hands-ons.]

We used a temporal train/test split approach where all the data from the 2024-25 season was used for the training set while the data from 2025-26 season was used for the test set. For feature engineering we focused on predicting next gameweek's points. We performed data cleaning to include important player positions which were 'GK' (GoalKeeper), 'DEF' (Defender), 'MID' (MidFielder) and 'FWD' (Forward/Attacker). We then sorted the dataset in chronological order by season, player name and gameweek so that the player's performance is organized over time and that we are using information from earlier gameweeks for predictions.

4.2 Model Training

[Describe the three (or more) ML models you implemented. Explain your use of N-fold cross-validation and your model selection strategy.]

We implemented 3 ML models:

Linear Regression - This is used as a simple baseline model that finds a linear relationship between features like past performance and player position. This model helps illustrate a simple baseline for how well the features capture player performance.

Decision Tree Regressor - This is used for non-linear patterns in player performance and this splits data based on different conditions like opponents strength or player position.

Random Forest Regressor - This is a more advanced model which combines multiple decision trees or complex interactions between features like player's role, and opponent strength. This results in more accurate predictions for next gameweek points.

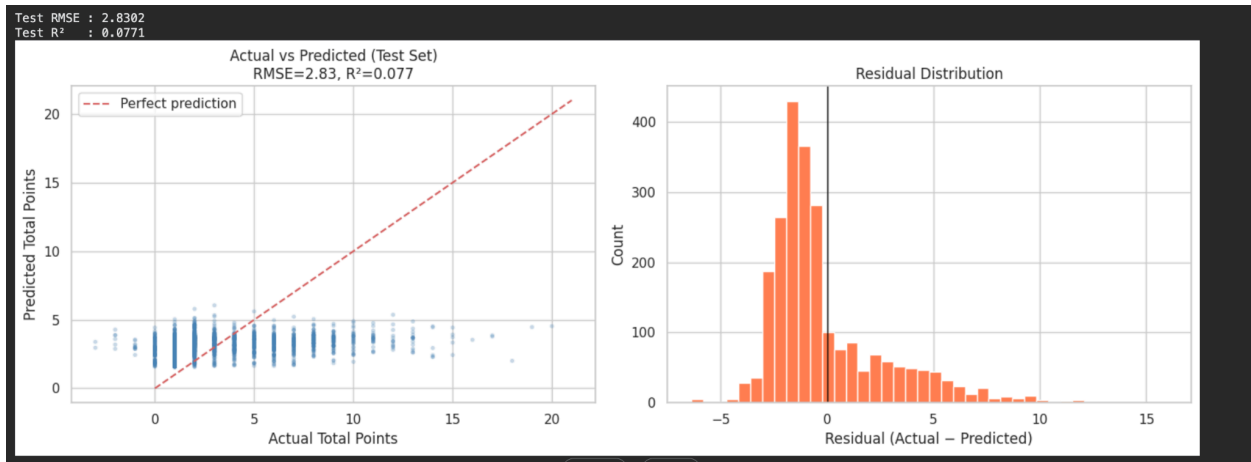
For each model these were integrated using scikit learn library tool along with preprocessing steps which we used for both training and evaluating the model. We used 5-fold cross validation on the training set (2024-25) where 4 folds were used for training and 1 fold was used for validation.

4.3 Model Evaluation & Testing

[Discuss the evaluation metrics you used (e.g., accuracy, precision, recall, F1, ROC, MSE). Compare the models clearly.]

Model	Model A CV RMSE	Model B CV RMSE	Notes
Linear Regression	2.820 ± 0.170	2.910 ± 0.198	Best CV in both models
Decision Tree	4.178 ± 0.120	4.382 ± 0.181	Performed the worst in both
Random Forest (default)	2.910 ± 0.166	3.004 ± 0.182	Improved after tuning
Random Forest (tuned)	2.828	2.921	Selected for final evaluation

Setup Train rows Test rows Features Tuned CV RMSE Test RMSE Test R²							
Model B (single-season) Train: 2025-26 GW1-21, Test: GW22-29							
	5343	2374	22	2.9206	2.8302	0.0771	



5. Results, Code Output & Interpretation

[Include at least one representative program output and explain it clearly. Provide screenshots of important code snippets. Connect your final ML results and visualization insights back to your

original 5 analytical questions.]

We trained two regression model variants, a cross season Model A and a single season Model B, both using Linear Regression, Decision Tree, and Random Forest.

- Includes one representative program output, explained clearly.

5.1 Representative Program Output

```
Test RMSE : 2.8302
Test R2   : 0.0771
```

The root mean square error for the model B was 2.8302 meaning that our predictions were off by about 2.8302 FPL points on average. The R^2 of 0.771 means that the model only explains a variance of 7.7% from the actual scores. While this is low it can be explained since the rolling average cannot anticipate the bonus points gained or lost through different scenarios such as scoring multiple goals or getting red cards.

5.2 Comparison between Models

	Setup	Train rows	Test rows	Features	Tuned	CV RMSE	Test RMSE	Test R ²
Model A (cross-season)	Train: 2024-25, Test: 2025-26	10307	7717	21		2.8283	2.9235	0.0521
Model B (single-season)	Train: 2025-26 GW1-21, Test: GW22-29	5343	2374	22		2.9206	2.8302	0.0771

Model A wins on cross-validation (2.83 vs 2.92) because it has roughly twice the training data, but Model B wins on the held-out test set (RMSE 2.83 vs 2.92, R^2 0.077 vs 0.052). The likely reason is distribution shift between seasons: tactics, scoring rules, and the new defensive scoring change (introduced in 2025-26) all mean the cross-season model trains on a slightly different game than it predicts on. Model B avoids that shift.

```

rolling_cols = [
    'total_points', 'minutes', 'goals_scored', 'assists',
    'bonus', 'ict_index', 'creativity', 'threat', 'influence',
    'xP', 'expected_goals', 'expected_assists',
    'clean_sheets', 'saves',
    'defensive_contribution'
]

for col in rolling_cols:
    if col in df.columns:
        df[f'_{col}_played'] = df[col].where(df['minutes'] > 0)

for col in rolling_cols:
    if col in df.columns:
        df[f'{col}_roll3'] = (
            df.groupby(['season', 'name'])[f'_{col}_played']
              .transform(lambda x: x.shift(1)
                          .rolling(3, min_periods=1)
                          .mean()
                          .ffill())
        )
        df[f'{col}_roll5'] = (
            df.groupby(['season', 'name'])[f'_{col}_played']
              .transform(lambda x: x.shift(1)
                          .rolling(5, min_periods=1)
                          .mean()
                          .ffill())
        )

df['points_lag1'] = (
    df.groupby(['season', 'name'])['total_points_played']
      .transform(lambda x: x.shift(1).ffill())
)

df.drop(columns=[f'_{col}_played' for col in rolling_cols if col in df.columns], inplace=True)
print('Done.')

... Done.

```

Code for Rolling columns

1. How well can we predict a player's next-game performance using historical statistics?

Predicting next-game performance. RMSE on the held-out test is ~ 2.83 , and R^2 is only 0.077.

The model lives in a "safe middle" range (~ 2 – 5 points): it cannot anticipate big hauls or disasters. Per-gameweek RMSE is stable across all 8 test gameweeks (no drift), but the predicted weekly mean stays in a narrow ~ 3.03 – 3.13 band while actuals swing 2.82–3.15. There's also a small positive bias in 6 of 8 gameweeks. The model is useful for ranking expected output, not for picking the captain pick that explodes for 20 points.

2. How do contextual factors such as home/away status, opponent difficulty, and player position affect player output?

Contextual factors (covered in P2). Home vs away has a small but visible effect (~ 0.2 pts on the median, larger on the upper tail). Opponent strength compresses the high end: against the strongest opponents, the 75th percentile of points drops sharply.

3. What combination of players is expected to maximize team performance under budget constraints?

Lineup optimization (covered in P3). The interactive Panel dashboard searches a simplified 5-player lineup under a budget cap, optimizing either total points or points-per-cost.

4. Which statistics are the strongest early indicators of a breakout player?

Breakout indicators. Playing time dominates Model B's importance ranking — minutes_roll3 (0.234) and minutes_roll5 (0.117) take the top two spots, because a player who is not on the pitch cannot score. After that, value (FPL price, 0.113) and ict_index_roll3 (0.089) rank #3 and #4. Surprisingly, total_points_roll5 is only mid-pack (~0.05): past points alone are not the strongest signal. The breakout profile is a player with consistent minutes, a rising ICT index, and a manageable opponent.

5. If a key player becomes unavailable, can we recommend the best replacement?

Replacement recommendation. Cosine similarity over per-player rolling-feature profiles surfaces strong style matches: e.g., the top training scorer Haaland (FWD, value 147.8, 139 pts) is paired with Hugo Ekitiké (similarity 0.995) and other forwards. Important caveat: similarity matches profile shape (minutes pattern, ICT involvement) not absolute output. For practical use, pair the similarity ranking with an absolute filter on output or price (the budget parameter does this).

6. Conclusion & Limitation

6.1 Conclusions:

- Model B was better at predicting the data compared to Model A due to the scoring changes due to the addition of the defensive contribution points, present in test for both and training for Model B alone.
- Players who played more tended to have a higher score predicted on average.
- The model identifies who would score, and predicts them to have a score between 0-6 regardless of how many they score.

6.2 Limitations:

- Schema drift between seasons. defensive_contribution, recoveries, tackles, and clearances_blocks_interceptions only exist in 2025-26, so Model A had to drop the new defensive feature and Model B had less training data.
- No injury, suspension, or starter context. A player who looked great in training may simply be on the bench next gameweek; the model has no way to know.
- Highly skewed target. ~95% of test rows score under 7 points, so the model converges on safe predictions in the 1–6 range. Linear regression is competitive with random forest on this target, which suggests the residual variance is mostly noise rather than complex non-linear structure.

- Tree-based regression cannot extrapolate. Random forest predictions are bounded by the training distribution, which is why the model never predicts above ~6 points even when an in-form player is about to score 12.
- The rolling columns average cannot anticipate if a player will get bonus points that could have their score way higher than normal or even below 0.
- Some players could start consistently performing after joining in the mid-season transfer or getting playtime due to injuries to the main players, but would be predicted to have lower scores due to the lack of playtime.

7. Reproducibility, GitHub, & Documentation

GitHub Repository Link: [CS133-DataVisualization/term-project-group-16: term-project-group-16 created by GitHub Classroom](#)

[Ensure your repository contains complete code, data files, notebooks, and app files. Verify that your code is documented with comments/docstrings and that the folder organization is clear.]

- GitHub repo includes complete files.
- README includes environment setup, requirements, and run instructions.
- Code is documented.
- Folder organization is clear.

8. References

Dataset: Fantasy Premier League

ML Libraries: pandas, numpy, seaborn, plotly.express, matplotlib, plotly.graph_objects, geopy, panel, itertools, sklearn